```
# EXPERIMENT NO. 8
# Application of Optical Flow for Real-Time Object Tracking
# and Motion Analysis
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from IPython.display import display, Video
# STEP 1: Upload Video
print("Please upload a video file (.mp4 / .avi / .mov)")
uploaded = files.upload()
video_path = next(iter(uploaded.keys()))
print("\nVideo uploaded successfully:", video_path)
# PART A: LUCAS-KANADE OPTICAL FLOW
# Shi-Tomasi Corner Detection + Lucas-Kanade
print("\n========================================")
print("LUCAS-KANADE OPTICAL FLOW")
print("========================================")
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise Exception("Error: Could not open video.")
# Read first frame
ret, old_frame = cap.read()
if not ret:
    raise Exception("Error: Could not read first frame.")
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
# Shi-Tomasi Corner Detection parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
```

```
# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
# Detect feature points
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)
# Create mask for trajectories
mask = np.zeros_like(old_frame)
# Video properties
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
if fps <= 0:
    fps = 20
# Output video
output_lk = "Lucas_Kanade_Optical_Flow.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out_lk = cv2.VideoWriter(
    output_lk,
    fourcc,
    fps,
    (width, height)
```

```
)
frame_count = 0
total_displacement = 0
total_points = 0
while True:
  ret, frame = cap.read()
   if not ret:
        break
 frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )
  # Check feature points
    if p0 is not None and len(p0) > 0:
  # Calculate Lucas-Kanade Optical Flow
        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )
   if p1 is not None:
  # Select successfully tracked points
            good_new = p1[st == 1]
            good_old = p0[st == 1]
 # Draw motion trajectories
  for new, old in zip(good_new, good_old):
               x_new, y_new = new.ravel()
               x_old, y_old = old.ravel()
  # Draw trajectory
               mask = cv2.line(
                   mask,
```

```
                    (int(x_old), int(y_old)),
                    (int(x_new), int(y_new)),
                    (0, 255, 0),
                    2
                )
                # Draw tracking point
                frame = cv2.circle(
                    frame,
                    (int(x_new), int(y_new)),4,(0, 0, 255), -1)
  # Calculate displacement
                displacement = np.sqrt(
                    (x_new - x_old) ** 2 +
                    (y_new - y_old) ** 2
                )
total_displacement += displacement
                total_points += 1
# Update points
            if len(good_new) > 0:
                p0 = good_new.reshape(-1, 1, 2)
            else:
                p0 = cv2.goodFeaturesToTrack(
                    frame_gray,
                    mask=None,
                    **feature_params
                )
 # Combine frame and trajectory
    output_frame = cv2.add(
        frame,
        mask
    )
 # Display title
    cv2.putText(
        output_frame,
```

```
        "Lucas-Kanade Optical Flow",
        (20, 35),
        cv2.FONT_HERSHEY_SIMPLEX,0.9, (255, 255, 255),2)

    # Save frame
    out_lk.write(output_frame)
    # Update previous frame
    old_gray = frame_gray.copy()
    frame_count += 1
out_lk.release()
cap.release()
# Calculate average displacement
if total_points > 0:
    average_displacement = (
        total_displacement / total_points
    )
else:
    average_displacement = 0
print("Lucas-Kanade tracking completed.")
print("Frames processed:", frame_count)
print(
    "Average displacement:",
    round(average_displacement, 2),
    "pixels/frame"
)
# PART B: FARNEBACK DENSE OPTICAL FLOW
print("\n========================================")
print("FARNEBACK DENSE OPTICAL FLOW")
print("========================================")
cap = cv2.VideoCapture(video_path)
ret, frame1 = cap.read()
if not ret:
    raise Exception("Error: Could not read video.")
```

```
prev_gray = cv2.cvtColor(
    frame1,
    cv2.COLOR_BGR2GRAY
)
# Output video
output_farneback = "Farneback_Optical_Flow.mp4"
out_fb = cv2.VideoWriter(
    output_farneback,
    fourcc,
    fps,
    (width, height)
)
frame_count_fb = 0
total_magnitude = 0
magnitude_count = 0
while True:
    ret, frame2 = cap.read()
    if not ret:
        break
    next_gray = cv2.cvtColor(
        frame2,
        cv2.COLOR_BGR2GRAY
    )
    # Calculate Farneback Dense Optical Flow
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,next_gray,None,0.5,3,15,3,5,1.2,0 )
 # Convert flow into magnitude and direction
    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )
# Calculate average motion
    avg_magnitude = np.mean(magnitude)
```

```
total_magnitude += avg_magnitude
magnitude_count += 1
# Create HSV image for optical flow visualization
hsv = np.zeros_like(frame2)
hsv[..., 1] = 255
# Motion direction
hsv[..., 0] = (
    angle * 180 / np.pi / 2
)
# Motion magnitude
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)
# Convert HSV to BGR
flow_visual = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)
# Add title
cv2.putText(
    flow_visual,
    "Farneback Dense Optical Flow",
    (20, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.9,
    (255, 255, 255),
    2
)
```

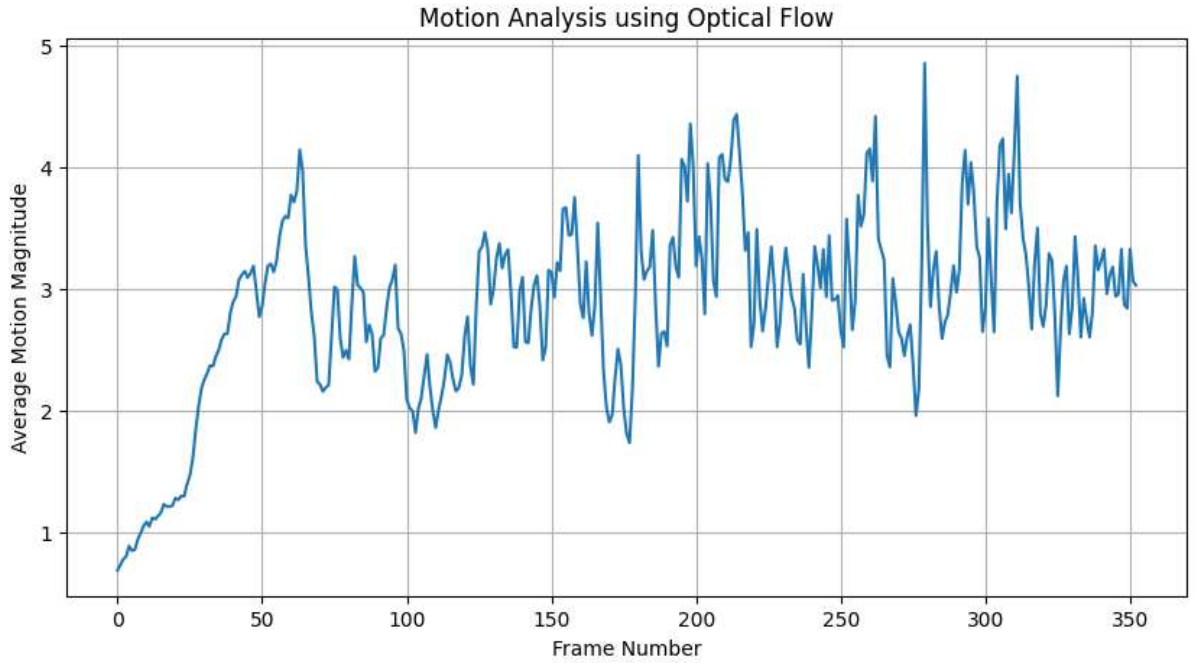
```
    # Display motion magnitude
    cv2.putText(
        flow_visual,
        f"Motion Magnitude: {avg_magnitude:.2f}",
        (20, 70),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )
    # Save output
    out_fb.write(flow_visual)
    # Update previous frame
    prev_gray = next_gray.copy()
    frame_count_fb += 1
out_fb.release()   cap.release()
# Calculate average flow magnitude
if magnitude_count > 0:
    average_flow = (
        total_magnitude / magnitude_count
    )
else:
    average_flow = 0
print("Farneback optical flow completed.")
print("Frames processed:", frame_count_fb)
print(  "Average motion magnitude:",round(average_flow, 2)
)
# PART C: MOTION ANALYSIS GRAPH
print("\n========================================")
print("MOTION ANALYSIS")
print("========================================")
cap = cv2.VideoCapture(video_path)
ret, prev_frame = cap.read()
```

```
if not ret:
    raise Exception("Error: Could not read video.")
prev_gray = cv2.cvtColor(prev_frame,cv2.COLOR_BGR2GRAY)
motion_values = []
frame_numbers = []
frame_no = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY
    )
 # Farneback Optical Flow
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray,None,  0.5,3,
15,3,5,1.2, 0)
 # Magnitude and direction
    magnitude, angle = cv2.cartToPolar(flow[..., 0],flow[..., 1])
    # Average motion magnitude
    avg_motion = np.mean(magnitude)
   motion_values.append(avg_motion)
    frame_numbers.append(frame_no)
    prev_gray = gray.copy()
    frame_no += 1
cap.release()
# Plot graph
plt.figure(figsize=(10, 5))
plt.plot(frame_numbers,  motion_values, linewidth=1.5)
plt.xlabel("Frame Number")
plt.ylabel("Average Motion Magnitude")
plt.title("Motion Analysis using Optical Flow")
plt.grid(True)
plt.show()
# FINAL RESULTS
```

```
print("\n========================================")
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("========================================")
print("\nResults:")
print("1. Lucas-Kanade Optical Flow")
print("2. Farneback Dense Optical Flow")
print("3. Motion Magnitude Analysis")
print("4. Object Trajectory Visualization")
print("\nAverage Lucas-Kanade Displacement:",
round(average_displacement, 2), "pixels/frame")
print("Average Farneback Motion Magnitude:",round(average_flow, 2))
# DISPLAY OUTPUT VIDEOS
print("\n========================================")
print("LUCAS-KANADE OUTPUT")
print("========================================")
display(Video(output_lk, embed=True))
print("\n========================================")
print("FARNEBACK OUTPUT")
print("========================================")
display(
    Video(
        output_farneback,
        embed=True
    )
)
# DOWNLOAD OUTPUT VIDEOS
print("\nDownloading output files...")
files.download(output_lk)
files.download(output_farneback)
print("\nAll files generated successfully!")
```

OUTPUT –

```
Video uploaded successfully: 16865051_1440_2562_30fps.mp4
LUCAS-KANADE OPTICAL FLOW
Lucas-Kanade tracking completed.
Frames processed: 353
Average displacement: 3.52 pixels/frame
FARNEBACK DENSE OPTICAL FLOW
Farneback optical flow completed.
Frames processed: 353
Average motion magnitude: 2.85
MOTION ANALYSIS
```




```
EXPERIMENT COMPLETED SUCCESSFULLY
Results:
1. Lucas-Kanade Optical Flow
2. Farneback Dense Optical Flow
3. Motion Magnitude Analysis
4. Object Trajectory Visualization
Average Lucas-Kanade Displacement: 3.52 pixels/frame
Average Farneback Motion Magnitude: 2.85
```